Eventos, Integrações e Arquitetura Desacoplada

Efeitos colaterais confiáveis sem acoplar o caso de uso

Treinamento prático com decisões técnicas, código executável e critérios de qualidade

Roteiro da aula

01	**Evento de domínio**	Representar um fato relevante do negócio
02	**Execução assíncrona**	Listeners, jobs, retries e idempotência
03	**Integrações confiáveis**	Contrato, webhook e outbox
04	**Projeto do dia**	Desacoplar a criação de agendamento

Efeitos colaterais no ClinicFlow

Criar um agendamento envia confirmação, atualiza o dashboard, sincroniza calendário e publica webhook. Uma falha externa não pode desfazer o agendamento.

ACOPLAMENTO ATUAL

- Action conhece todos os provedores
- latência cresce a cada integração
- retry repete operações internas

RESULTADO DESEJADO

- transação curta
- efeitos rastreáveis
- consumidores independentes

Evento de domínio imutável

O evento registra um fato no passado e transporta identificadores estáveis, não um model mutável carregado.

```
final readonly class AppointmentCreated
    implements ShouldDispatchAfterCommit
{
    use Dispatchable;

    public function __construct(
        public int $appointmentId,
        public int $clinicId,
        public CarbonImmutable $occurredAt,
    ) {}
}
```

Nome e payload

- fato no passado
- dados mínimos
- tenant explícito
- versão quando necessário

O evento comunica que algo ocorreu. Ele não ordena como cada consumidor deve reagir.

Listener e job enfileirado

O listener traduz o fato em trabalho assíncrono. O job remove a latência externa da requisição principal.

```
final class QueueAppointmentConfirmation
{
    public function handle(AppointmentCreated $event): void
    {
        SendAppointmentConfirmation::dispatch(
            $event->appointmentId,
            $event->clinicId,
        )->onQueue('notifications');
    }
}
```

Contrato do job

- retry seguro
- timeout definido
- fila específica
- falha observável

O listener organiza a reação. O job executa o trabalho demorado e precisa ser idempotente.

Evento após commit

Ao implementar ShouldDispatchAfterCommit, o Laravel só publica o evento quando a transação confirma.

```
DB::transaction(function () use ($data) {
    $appointment = Appointment::create($data);

    AppointmentCreated::dispatch(
        $appointment->id,
        $appointment->clinic_id,
        now()->toImmutable(),
    );
});
```

Garantia

- ShouldDispatchAfterCommit
- não publique em rollback
- payload com IDs
- consumidor tolera atraso

A garantia evita consumo de dados não confirmados, mas não cobre falha do processo. Para isso, use outbox.

Camada anticorrupção

O domínio depende de um contrato próprio. O adapter traduz modelos e erros do provedor externo.

```
interface ExternalCalendar
{
    public function create(CalendarAppointment $appointment): ExternalEventId;
}

final class GoogleCalendarAdapter implements ExternalCalendar
{
    public function create(CalendarAppointment $appointment): ExternalEventId
    {
        return $this->mapper->fromResponse($this->client->insert($appointment));
    }
}
```

Benefício

- SDK fica no adapter
- erros são traduzidos
- teste usa fake
- troca de provedor é localizada

O contrato deve refletir a necessidade do domínio, não copiar a API externa.

Integração resiliente

Timeout, retry com backoff e chave de idempotência tornam a falha externa controlável e observável.

```
$response = Http::acceptJson()
    ->timeout(5)
    ->retry([200, 500, 1000], throw: false)
    ->withHeaders(['Idempotency-Key' => $delivery->uuid])
    ->post($endpoint, $payload);

$response->throw();
```

Respostas HTTP

- 4xx não transitório
- 429 respeita Retry-After
- 5xx pode repetir
- timeout é resultado desconhecido

Retry sem idempotência pode duplicar compromissos no sistema parceiro.

Webhook com entrega rastreável

Crie uma entrega persistida antes do envio. O job atualiza tentativa, resposta e próxima execução.

```
$delivery = WebhookDelivery::firstOrCreate(
    ['event_id' => $eventId, 'endpoint_id' => $endpoint->id],
    ['status' => DeliveryStatus::Pending],
);

DeliverWebhook::dispatch($delivery->id);
```

Operação

- chave única
- assinatura HMAC
- backoff
- dead-letter e replay

O parceiro externo não participa da transação que cria o agendamento.

Outbox na mesma transação

O registro de negócio e a intenção de publicar são gravados juntos. Um publisher independente entrega o evento.

```
DB::transaction(function () use ($data) {
    $appointment = Appointment::create($data);

    OutboxMessage::create([
        'event_id' => (string) Str::uuid(),
        'type' => 'appointment.created.v1',
        'payload' => ['appointment_id' => $appointment->id],
    ]);
});
```

Use quando

- perda de evento é crítica
- há múltiplos consumidores
- replay é necessário
- operação aceita consistência eventual

Outbox aumenta complexidade operacional. Aplique quando o risco justificar publisher, retenção e monitorização.

Observabilidade do fluxo assíncrono

O evento precisa ser rastreável do caso de uso até cada efeito colateral.

Correlação

- event_id e request_id
- clinic_id
- causation_id

Métricas

- idade da mensagem
- taxa de retry
- entregas falhadas

Recuperação

- replay controlado
- idempotência
- dead-letter documentada

Um fluxo desacoplado sem visibilidade apenas distribui a dificuldade de diagnóstico.

Projeto do dia: criação desacoplada

Missão

Separar os efeitos colaterais da criação de agendamento e manter entregas externas recuperáveis.

Entregas

- AppointmentCreated imutável
- listeners e jobs enfileirados
- ExternalCalendar por contrato
- WebhookDelivery persistida

Aceite

- request não aguarda provedor
- retry não duplica efeito
- event_id aparece nos logs
- falha pode ser reprocessada

A apresentação da equipe deve ligar problema, decisão, evidência e resultado.

Checklist de integração confiável

1	**Registrar**	evento representa um fato confirmado
2	**Separar**	listener traduz fato em trabalho
3	**Proteger**	timeout, retry e idempotência
4	**Rastrear**	event_id liga produtor e consumidores
5	**Recuperar**	replay seguro e falha visível

Saída esperada: efeitos colaterais independentes sem perda silenciosa nem duplicação.